COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 212 - Automating Server Management	**SECTION:**	CPE31S2
DATE PERFORMED:	10/2/2026	**DATE SUBMITTED:**	10/4/2026
NAME:	Charles Vence Espinas	**INSTRUCTOR:**	Engr. Jimlord Quejado

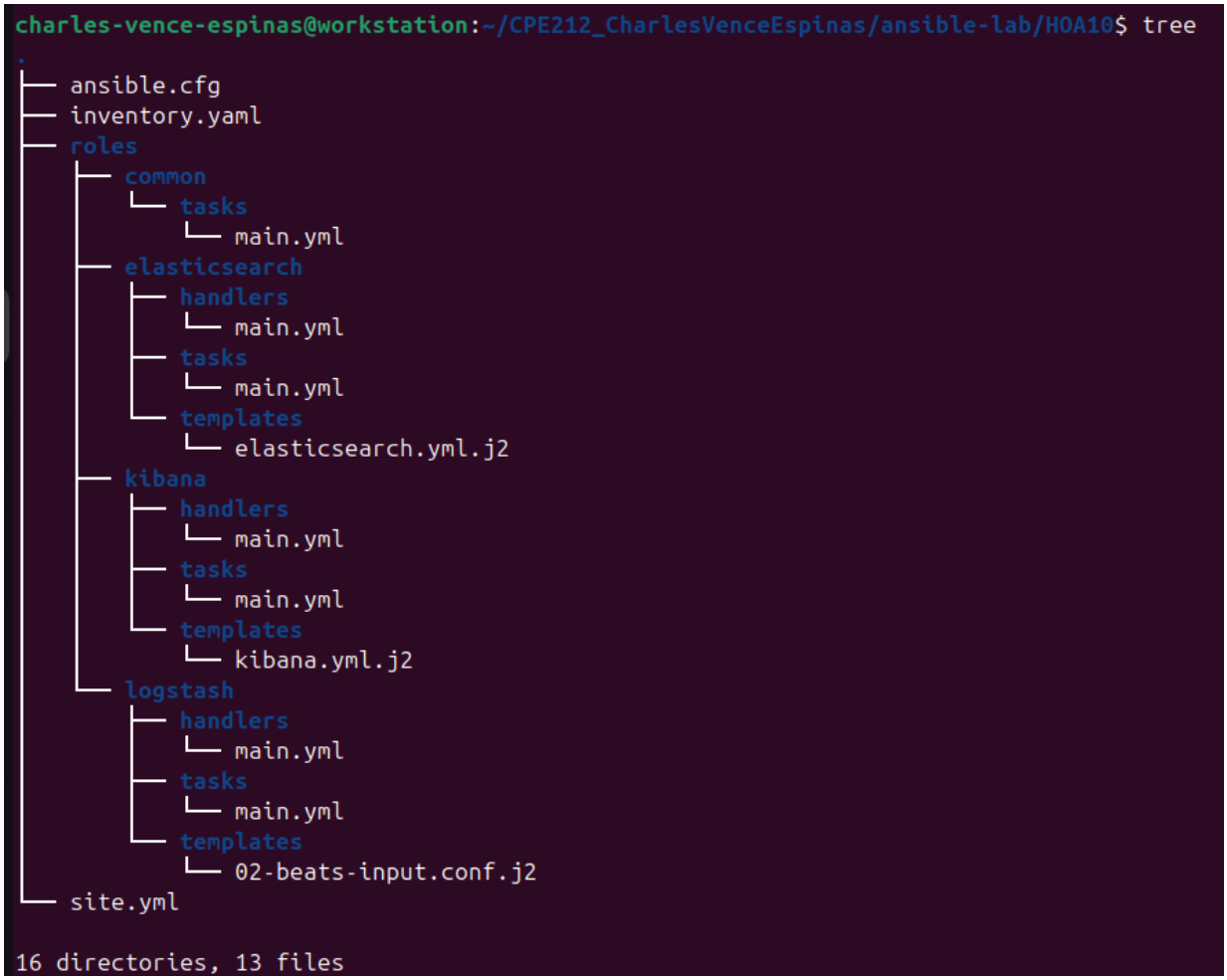
ACTIVITY NO. 10 **Install, Configure and Manage Enterprise Log Monitoring via Ansible**

1. PROCEDURE OUTPUT

GITHUB LINK: https://github.com/charlesespinas/CPE212_CharlesVenceEspinas

1. Create a playbook that:
   a. Install and configure Elastic Stack in separate hosts (Elastic Search, Kibana, Logstash)
2. Apply the concept of creating roles.
3. Describe how you did step 1. (Provide screenshots and explanations in your report. Make your report detailed such that it will look like a manual.)
4. Show an output of the installed Elastic Stack for both Ubuntu and CentOS.
5. Make sure to create a new repository in GitHub for this activity.

File Structure



- The directory tree shows a modular Ansible role structure which is designed for easy management and future expansion. The setup separates the basic server configuration into the common role while Elasticsearch, Logstash, and Kibana have their own dedicated role directories which contain task definitions, Jinja2 dynamic templates, and systemd handlers. The main configuration is managed through ansible.cfg and inventory.yaml, while site.yml serves as the main playbook which runs and manages all the roles.

ansible.cfg

```
 GNU nano 7.2                                          ansible.cfg
[defaults]
inventory = inventory.yaml
host_key_checking = False
roles_path = ./roles
```

- The ansible.cfg file defines the main settings for the Ansible control node. It automatically uses inventory.yaml as the default host file and sets ./roles as the directory where Ansible looks for role dependencies. Setting host_key_checking = False skips the initial SSH fingerprint prompts which allows playbooks to run automatically without requiring manual confirmation when connecting to newly installed VMs.

inventory.yaml

```
 GNU nano 7.2                                          inventory.yaml
all:
  vars:
    ansible_user: charles-vence-espinas

  children:
    elasticsearch_node:
      hosts:
        server1:
          ansible_host: 192.168.56.108

    logstash_node:
      hosts:
        server2:
          ansible_host: 192.168.56.109

    kibana_node:
      hosts:
        centOS:
          ansible_host: 192.168.56.104

    elk_stack:
      children:
        elasticsearch_node:
        logstash_node:
        kibana_node:
```

- The inventory.yaml file defines the target machines for deploying the multi node Elastic Stack. The global settings like the SSH execution user charles-vence-espinas are applied to all nodes. Each host group (elasticsearch_node, logstash_node, and kibana_node) is assigned to its respective target IP address

(192.168.56.108, 192.168.56.109, and 192.168.56.104). The parent group (elk_stack) contains all the nodes which allows basic configuration tasks to be applied to all machines at the same time.

site.yml

```
 GNU nano 7.2                                                site.yml
---
- name: Apply common configurations across all ELK nodes
  hosts: elk_stack
  become: true
  roles:
    - common

- name: Install and configure Elasticsearch
  hosts: elasticsearch_node
  become: true
  roles:
    - elasticsearch

- name: Install and configure Logstash
  hosts: logstash_node
  become: true
  roles:
    - logstash

- name: Install and configure Kibana
  hosts: kibana_node
  become: true
  roles:
    - kibana
```

- The site.yml playbook serves as the main execution workflow which connects the infrastructure groups to their assigned Ansible roles. It runs sequentially across all target hosts using elevated privileges (become: true). First, the basic settings are applied to all nodes under elk_stack through the common role, followed by separate plays which install and configure Elasticsearch on Ubuntu (server1), Logstash on Ubuntu (server2), and Kibana on CentOS (centOS).

roles/common/tasks/main.yml

GNU nano 7.2 main.yml

```yaml
---
- name: Create swap file on low-memory nodes (<= 2GB RAM)
  block:
    - name: Create 2GB swap file
      command: fallocate -l 2G /swapfile
      args:
        creates: /swapfile

    - name: Set correct permissions on swap file
      file:
        path: /swapfile
        owner: root
        group: root
        mode: '0600'

    - name: Make swap filesystem
      command: mkswap /swapfile
      register: mkswap_result
      changed_when: "'setting up swapspace' in mkswap_result.stdout"
      failed_when: false

    - name: Enable swap
      command: swapon /swapfile
      register: swapon_result
      changed_when: "swapon_result.rc == 0"
      failed_when: false
```

```
GNU nano 7.2                                                main.yml
    - name: Ensure swap is mounted on boot in fstab
      mount:
        name: none
        src: /swapfile
        fstype: swap
        opts: sw
        state: present
  when: ansible_memtotal_mb <= 2048

- name: Install prerequisites on Debian/Ubuntu
  apt:
    name:
      - apt-transport-https
      - gnupg2
      - openjdk-11-jre
    state: present
    update_cache: yes
  when: ansible_os_family == "Debian"

- name: Add Elastic GPG key on Debian/Ubuntu
  apt_key:
    url: https://artifacts.elastic.co/GPG-KEY-elasticsearch
    state: present
  when: ansible_os_family == "Debian"
```

```
- name: Add Elastic repository on Debian/Ubuntu
  apt_repository:
    repo: "deb https://artifacts.elastic.co/packages/8.x/apt stable main"
    state: present
    filename: elastic-8.x
  when: ansible_os_family == "Debian"

- name: Import Elastic GPG key on RHEL/CentOS
  rpm_key:
    key: https://artifacts.elastic.co/GPG-KEY-elasticsearch
    state: present
  when: ansible_os_family == "RedHat"

- name: Add Elastic repository on RHEL/CentOS
  yum_repository:
    name: elastic-8.x
    description: Elastic repository for 8.x packages
    baseurl: https://artifacts.elastic.co/packages/8.x/yum
    gpgcheck: yes
    gpgkey: https://artifacts.elastic.co/GPG-KEY-elasticsearch
    enabled: yes
  when: ansible_os_family == "RedHat"
```

- The common task role prepares all Linux hosts with the required basic settings. A conditional block checks which nodes have 2GB RAM or less and creates a 2GB /swapfile with strict 0600 root permissions which helps prevent Out-Of-Memory (OOM) kernel kills while running the Elastic Stack. The OS-family conditions (ansible_os_family) then configure the official Elastic 8.x package repositories and GPG signing keys. It uses apt for Debian/Ubuntu nodes (server1, server2) and yum/rpm for RedHat/CentOS nodes (centOS).

roles/elasticsearch/handlers/main.yml

```
 GNU nano 7.2                                  roles/elasticsearch/handlers/main.yml
---
- name: restart elasticsearch
  systemd:
    name: elasticsearch
    state: restarted
```

- This handler task monitors configuration changes made by the elasticsearch role. When files like elasticsearch.yml are modified, the handler restarts the service through systemd which allows the changes to take effect without running the service unnecessarily when no changes are made.

roles/elasticsearch/tasks/main.yml

```
  GNU nano 7.2                                  roles/elasticsearch/tasks/main.yml
---
- name: Install Elasticsearch
  package:
    name: elasticsearch
    state: present

- name: Ensure Elasticsearch data and log directories exist
  file:
    path: "{{ item }}"
    state: directory
    owner: elasticsearch
    group: elasticsearch
    mode: '0755'
  loop:
    - /var/lib/elasticsearch
    - /var/log/elasticsearch

- name: Limit JVM heap size to 512MB
  copy:
    dest: /etc/elasticsearch/jvm.options.d/heap.options
    content: |
      -Xms512m
      -Xmx512m
    owner: elasticsearch
    group: elasticsearch
    mode: '0644'
```

```yaml
- name: Configure elasticsearch.yml
  template:
    src: elasticsearch.yml.j2
    dest: /etc/elasticsearch/elasticsearch.yml
    owner: elasticsearch
    group: elasticsearch
    mode: '0660'
  notify: restart elasticsearch

- name: Enable and start Elasticsearch service
  systemd:
    name: elasticsearch
    enabled: yes
    state: started

- name: Allow Elasticsearch port 9200 on Debian/Ubuntu (UFW)
  ufw:
    rule: allow
    port: '9200'
    proto: tcp
    state: enabled
  when: ansible_os_family == "Debian"
```

```yaml
- name: Allow Elasticsearch port 9200 on RHEL/CentOS (firewalld)
  firewalld:
    port: 9200/tcp
    permanent: yes
    state: enabled
    immediate: yes
  when: ansible_os_family == "RedHat"
```

- The elasticsearch role task manages the package installation, directory setup, memory allocation, and network service settings. It sets both the minimum and maximum JVM heap allocation to 512MB (-Xms512m, -Xmx512m) which helps maintain service stability when using limited hardware resources. It also allows incoming connections through port 9200/tcp by using ufw on Debian/Ubuntu and firewalld on RedHat/CentOS which ensures that the service can be accessed across different operating systems.

roles/elasticsearch/templates/elasticsearch.yml.j2

```
  GNU nano 7.2                          roles/elasticsearch/templates/elasticsearch.yml.j2
cluster.name: enterprise-elk-cluster
node.name: node-1

path.data: /var/lib/elasticsearch
path.logs: /var/log/elasticsearch

network.host: 0.0.0.0
http.port: 9200
discovery.type: single-node

xpack.security.enabled: false
xpack.security.enrollment.enabled: false
xpack.security.http.ssl.enabled: false
xpack.security.transport.ssl.enabled: false
```

- This Jinja2 template configures the cluster settings and simplifies the parameters for single-node deployment. Setting network.host: 0.0.0.0 allows Elasticsearch to listen on all active network interfaces which enables remote connections through port 9200. X-Pack security features and SSL authentication are also disabled which allows direct communication between Logstash, Kibana, and Elasticsearch in the lab environment.

roles/kibana/handlers/main.yml

```
  GNU nano 7.2                                    roles/kibana/handlers/main.yml
---
- name: restart kibana
  systemd:
    name: kibana
    state: restarted
```

- This handler manages service restarts for Kibana whenever its main configuration file (kibana.yml) is modified by Ansible template tasks which allows the updated settings to take effect.

roles/kibana/tasks/main.yml

```
  GNU nano 7.2                                  roles/kibana/tasks/main.yml
---
- name: Install Kibana
  package:
    name: kibana
    state: present

- name: Configure kibana.yml
  template:
    src: kibana.yml.j2
    dest: /etc/kibana/kibana.yml
    owner: root
    group: kibana
    mode: '0660'
  notify: restart kibana

- name: Enable and start Kibana service
  systemd:
    name: kibana
    enabled: yes
    state: started

- name: Allow Kibana port 5601 on CentOS firewall
  firewalld:
    port: 5601/tcp
    permanent: yes
    state: enabled
    immediate: yes
  when: ansible_os_family == "RedHat"
```

- This role automates the installation and configuration of the Kibana web visualization application on CentOS (kibana_node). The playbook installs the kibana package, updates its main configuration file using templates, and enables systemd to keep the service running. On RedHat-family systems, firewalld rules are also configured to permanently allow incoming TCP traffic on port 5601 which allows the host workstation to access Kibana.

roles/kibana/templates/kibana.yml.j2

```
  GNU nano 7.2                            roles/kibana/templates/kibana.yml.j2
server.port: 5601
server.host: "0.0.0.0"
elasticsearch.hosts: ["http://{{ hostvars[groups['elasticsearch_node'][0]]['ansible_host'] }}:9200"]
```

- This template uses Jinja2 facts to allow Kibana to listen on all network interfaces (0.0.0.0:5601) which enables communication between different nodes. It also dynamically gets the IP address of the first host in the elasticsearch_node inventory group (192.168.56.108) which allows Kibana to send backend API requests to the Elasticsearch master node without manually setting the IP address.

roles/logstash/handlers/main.yml

```
 GNU nano 7.2                                  roles/logstash/handlers/main.yml
---
- name: restart logstash
  systemd:
    name: logstash
    state: restarted
```

- This handler restarts the Logstash service through systemd whenever pipeline configuration files inside /etc/logstash/conf.d/ are modified which allows the updated configurations to take effect.

roles/logstash/tasks/main.yml

```
 GNU nano 7.2                                    roles/logstash/tasks/main.yml
---
- name: Install Logstash
  package:
    name: logstash
    state: present

- name: Configure Logstash beats input pipeline
  template:
    src: 02-beats-input.conf.j2
    dest: /etc/logstash/conf.d/02-beats-input.conf
    owner: logstash
    group: logstash
    mode: '0644'
  notify: restart logstash

- name: Enable and start Logstash service
  systemd:
    name: logstash
    enabled: yes
    state: started
```

- This role installs and configures Logstash on server2 which handles log ingestion and pipeline processing. It installs the logstash package, places the pipeline configuration files inside /etc/logstash/conf.d/, sets the file permissions for the logstash user, and ensures that the service starts automatically when the system boots.

roles/logstash/templates/02-beats-input.conf.j2

```
  GNU nano 7.2                          roles/logstash/templates/02-beats-input.conf.j2
input {
  beats {
    port => 5044
  }
}

output {
  elasticsearch {
    hosts => ["http://{{ hostvars[groups['elasticsearch_node'][0]]['ansible_host'] }}:9200"]
    index => "logstash-%{+YYYY.MM.dd}"
  }
}
```

- This template configures the main Logstash data pipeline. The input stage opens TCP port 5044 to receive data from incoming Beats agents, while the output stage uses dynamic Ansible fact lookup to send the processed events to Elasticsearch (http://192.168.56.108:9200) which stores the data using daily index names (logstash-YYYY.MM.dd).

Testing Connectivity

```
charles-vence-espinas@workstation:~/CPE212_CharlesVenceEspinas/ansible-lab/HOA10$ ansible all -i inventory.yaml -m ping

server1 | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3"
    },
    "changed": false,
    "ping": "pong"
}
server2 | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3"
    },
    "changed": false,
    "ping": "pong"
}
centOS | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3"
    },
    "changed": false,
    "ping": "pong"
```

- The ad-hoc ansible ping command checks the network connection and authentication of all hosts listed in inventory.yaml before applying the configurations. This command verifies the host-only adapter network connections (192.168.56.108, 192.168.56.109, 192.168.56.104), SSH key authentication for the user charles-vence-espinas, and Python execution on the target hosts. Receiving a SUCCESS response with "ping": "pong" from each node confirms that all hosts are ready for configuration.

Output

```
charles-vence-espinas@workstation:~/CPE212_CharlesVenceEspinas/ansible-lab/HOA10$ ansible-playbook -i inventory.yaml sit
e.yml --ask-become-pass
BECOME password:

PLAY [Apply common configurations across all ELK nodes] ************************************************************

TASK [Gathering Facts] *********************************************************************************************
ok: [server2]
ok: [server1]
ok: [centOS]

TASK [common : Create 2GB swap file] *******************************************************************************
ok: [server1]
ok: [server2]
ok: [centOS]

TASK [common : Set correct permissions on swap file] ***************************************************************
ok: [server1]
ok: [server2]
ok: [centOS]

TASK [common : Make swap filesystem] *******************************************************************************
ok: [server1]
ok: [server2]
ok: [centOS]

TASK [common : Enable swap] ****************************************************************************************
ok: [server2]
ok: [server1]
ok: [centOS]
```

```
TASK [common : Ensure swap is mounted on boot in fstab] ************************************************************
ok: [server2]
ok: [server1]
ok: [centOS]

TASK [common : Install prerequisites on Debian/Ubuntu] *************************************************************
skipping: [centOS]
ok: [server1]
ok: [server2]

TASK [common : Add Elastic GPG key on Debian/Ubuntu] ***************************************************************
skipping: [centOS]
ok: [server2]
ok: [server1]

TASK [common : Add Elastic repository on Debian/Ubuntu] ************************************************************
skipping: [centOS]
ok: [server1]
ok: [server2]

TASK [common : Import Elastic GPG key on RHEL/CentOS] **************************************************************
skipping: [server1]
skipping: [server2]
ok: [centOS]

TASK [common : Add Elastic repository on RHEL/CentOS] **************************************************************
skipping: [server1]
skipping: [server2]
ok: [centOS]
```

```
PLAY [Install and configure Elasticsearch] *************************************************************************

TASK [Gathering Facts] *********************************************************************************************
ok: [server1]

TASK [elasticsearch : Install Elasticsearch] ***********************************************************************
ok: [server1]

TASK [elasticsearch : Ensure Elasticsearch data and log directories exist] *****************************************
ok: [server1] => (item=/var/lib/elasticsearch)
ok: [server1] => (item=/var/log/elasticsearch)

TASK [elasticsearch : Limit JVM heap size to 512MB] ****************************************************************
ok: [server1]

TASK [elasticsearch : Configure elasticsearch.yml] *****************************************************************
ok: [server1]

TASK [elasticsearch : Enable and start Elasticsearch service] ******************************************************
ok: [server1]

TASK [elasticsearch : Allow Elasticsearch port 9200 on Debian/Ubuntu (UFW)] ****************************************
ok: [server1]

TASK [elasticsearch : Allow Elasticsearch port 9200 on RHEL/CentOS (firewalld)] ************************************
skipping: [server1]
```

```
PLAY [Install and configure Logstash] ******************************************************************************

TASK [Gathering Facts] *********************************************************************************************
ok: [server2]

TASK [logstash : Install Logstash] *********************************************************************************
ok: [server2]

TASK [logstash : Configure Logstash beats input pipeline] **********************************************************
ok: [server2]

TASK [logstash : Enable and start Logstash service] ****************************************************************
ok: [server2]

PLAY [Install and configure Kibana] ********************************************************************************

TASK [Gathering Facts] *********************************************************************************************
ok: [centOS]

TASK [kibana : Install Kibana] *************************************************************************************
ok: [centOS]

TASK [kibana : Configure kibana.yml] *******************************************************************************
ok: [centOS]

TASK [kibana : Enable and start Kibana service] ********************************************************************
ok: [centOS]

TASK [kibana : Allow Kibana port 5601 on CentOS firewall] **********************************************************
ok: [centOS]
```

```
PLAY RECAP *********************************************************************************************************
centOS                     : ok=13   changed=0    unreachable=0    failed=0    skipped=3    rescued=0    ignored=0
server1                    : ok=16   changed=0    unreachable=0    failed=0    skipped=3    rescued=0    ignored=0
server2                    : ok=13   changed=0    unreachable=0    failed=0    skipped=2    rescued=0    ignored=0
```

- The ansible-playbook command runs site.yml which manages the deployment of Ansible roles across all target hosts in sequence. The --ask-become-pass flag asks for the administrative password which is required to gain root privileges on the target hosts. During execution, Ansible applies the swap configuration, installs packages, starts services, and configures firewall rules across all nodes. The process ends with a PLAY RECAP showing failed=0 which confirms that the playbook completed successfully

without any errors.

Verification

server1

```
charles-vence-espinas@server1:~$ sudo systemctl status elasticsearch
[sudo] password for charles-vence-espinas:
● elasticsearch.service - Elasticsearch
     Loaded: loaded (/usr/lib/systemd/system/elasticsearch.service; enabled; pr>
     Active: active (running) since Fri 2026-10-02 11:41:43 UTC; 15min ago
       Docs: https://www.elastic.co
   Main PID: 19949 (java)
      Tasks: 85 (limit: 1374)
     Memory: 740.3M (peak: 990.1M swap: 292.3M swap peak: 292.3M)
        CPU: 39.869s
     CGroup: /system.slice/elasticsearch.service
             ├─19949 /usr/share/elasticsearch/jdk/bin/java -Xms4m -Xmx64m -XX:+>
             ├─20006 /usr/share/elasticsearch/jdk/bin/java -Des.networkaddress.>
             └─20028 /usr/share/elasticsearch/modules/x-pack-ml/platform/linux->

Oct 02 11:41:24 server1 systemd[1]: Starting elasticsearch.service - Elasticsea>
Oct 02 11:41:43 server1 systemd[1]: Started elasticsearch.service - Elasticsear>
lines 1-15/15 (END)
```

```
charles-vence-espinas@server1:~$ curl -X GET http://localhost:9200
{
  "name" : "node-1",
  "cluster_name" : "enterprise-elk-cluster",
  "cluster_uuid" : "ScdQBotQR4qx3q2tzo9QXg",
  "version" : {
    "number" : "8.19.22",
    "build_flavor" : "default",
    "build_type" : "deb",
    "build_hash" : "3b2a41103de35e0af4064d647974032fcc1bcde9",
    "build_date" : "2026-09-18T10:10:07.018982263Z",
    "build_snapshot" : false,
    "lucene_version" : "9.12.2",
    "minimum_wire_compatibility_version" : "7.17.0",
    "minimum_index_compatibility_version" : "7.0.0"
  },
  "tagline" : "You Know, for Search"
}
```

- Verification on server1 (Ubuntu) checks the service status and REST API connection of the database tier. The systemctl status command confirms that the elasticsearch systemd service is active and running. Running curl -X GET http://localhost:9200 also confirms that Elasticsearch is listening on port 9200 and returning cluster information which includes the cluster name enterprise-elk-cluster and Elastic 8.x build details.

server2

```
charles-vence-espinas@server2:~$ sudo systemctl status logstash
[sudo] password for charles-vence-espinas:
● logstash.service - logstash
     Loaded: loaded (/usr/lib/systemd/system/logstash.service; enabled; preset:>
     Active: active (running) since Fri 2026-10-02 11:42:49 UTC; 18min ago
   Main PID: 18282 (java)
      Tasks: 47 (limit: 1374)
     Memory: 775.6M (peak: 853.2M swap: 12.9M swap peak: 12.9M)
        CPU: 45.623s
     CGroup: /system.slice/logstash.service
             └─18282 /usr/share/logstash/jdk/bin/java -Xms1g -Xmx1g -Djava.awt.>

Oct 02 11:59:20 server2 logstash[18282]: [2026-10-02T11:59:20,460][WARN ][logst>
Oct 02 11:59:31 server2 logstash[18282]: [2026-10-02T11:59:31,521][WARN ][logst>
Oct 02 11:59:42 server2 logstash[18282]: [2026-10-02T11:59:42,579][WARN ][logst>
Oct 02 11:59:54 server2 logstash[18282]: [2026-10-02T11:59:54,300][WARN ][logst>
Oct 02 12:00:05 server2 logstash[18282]: [2026-10-02T12:00:05,813][WARN ][logst>
Oct 02 12:00:17 server2 logstash[18282]: [2026-10-02T12:00:17,007][WARN ][logst>
Oct 02 12:00:28 server2 logstash[18282]: [2026-10-02T12:00:28,210][WARN ][logst>
Oct 02 12:00:39 server2 logstash[18282]: [2026-10-02T12:00:39,273][WARN ][logst>
Oct 02 12:00:50 server2 logstash[18282]: [2026-10-02T12:00:50,327][WARN ][logst>
Oct 02 12:01:01 server2 logstash[18282]: [2026-10-02T12:01:01,398][WARN ][logst>
lines 1-20/20 (END)
```

```
charles-vence-espinas@server2:~$ curl -I http://192.168.56.108:9200
HTTP/1.1 200 OK
X-elastic-product: Elasticsearch
content-type: application/json
content-length: 541
```

- Verification on server2 (Ubuntu) checks the Logstash service status and network connection to the database node. The systemctl status command confirms that the logstash service is active and running. Running curl -I http://192.168.56.108:9200 tests the network connection from server2 to server1 which returns an HTTP/1.1 200 OK response. This confirms that the Ubuntu UFW firewall rules allow Logstash to connect to Elasticsearch through port 9200 and forward log data.

centOS

```
[charles-vence-espinas@centOS ~]$ sudo systemctl status kibana
[sudo] password for charles-vence-espinas:
● kibana.service - Kibana
     Loaded: loaded (/usr/lib/systemd/system/kibana.service; enabled; preset:>
     Active: active (running) since Fri 2026-10-02 07:49:03 EDT; 13min ago
       Docs: https://www.elastic.co
   Main PID: 10841 (MainThread)
      Tasks: 11 (limit: 10189)
     Memory: 530.3M (peak: 906.7M)
        CPU: 29.513s
     CGroup: /system.slice/kibana.service
             └─10841 /usr/share/kibana/bin/../node/glibc-217/bin/node /usr/sh>

Oct 02 07:49:18 centOS kibana[10841]: [2026-10-02T07:49:18.505-04:00][INFO ][>
Oct 02 07:49:18 centOS kibana[10841]: [2026-10-02T07:49:18.829-04:00][WARN ][>
Oct 02 07:49:18 centOS kibana[10841]: [2026-10-02T07:49:18.829-04:00][INFO ][>
Oct 02 07:49:22 centOS kibana[10841]: [2026-10-02T07:49:22.508-04:00][WARN ][>
Oct 02 07:49:22 centOS kibana[10841]: [2026-10-02T07:49:22.509-04:00][INFO ][>
Oct 02 07:49:24 centOS kibana[10841]: [2026-10-02T07:49:24.581-04:00][INFO ][>
Oct 02 07:49:26 centOS kibana[10841]: [2026-10-02T07:49:26.118-04:00][INFO ][>
Oct 02 07:49:26 centOS kibana[10841]: [2026-10-02T07:49:26.119-04:00][INFO ][>
Oct 02 07:49:28 centOS kibana[10841]: [2026-10-02T07:49:28.391-04:00][INFO ][>
Oct 02 07:49:46 centOS kibana[10841]: [2026-10-02T07:49:46.750-04:00][ERROR][>
```

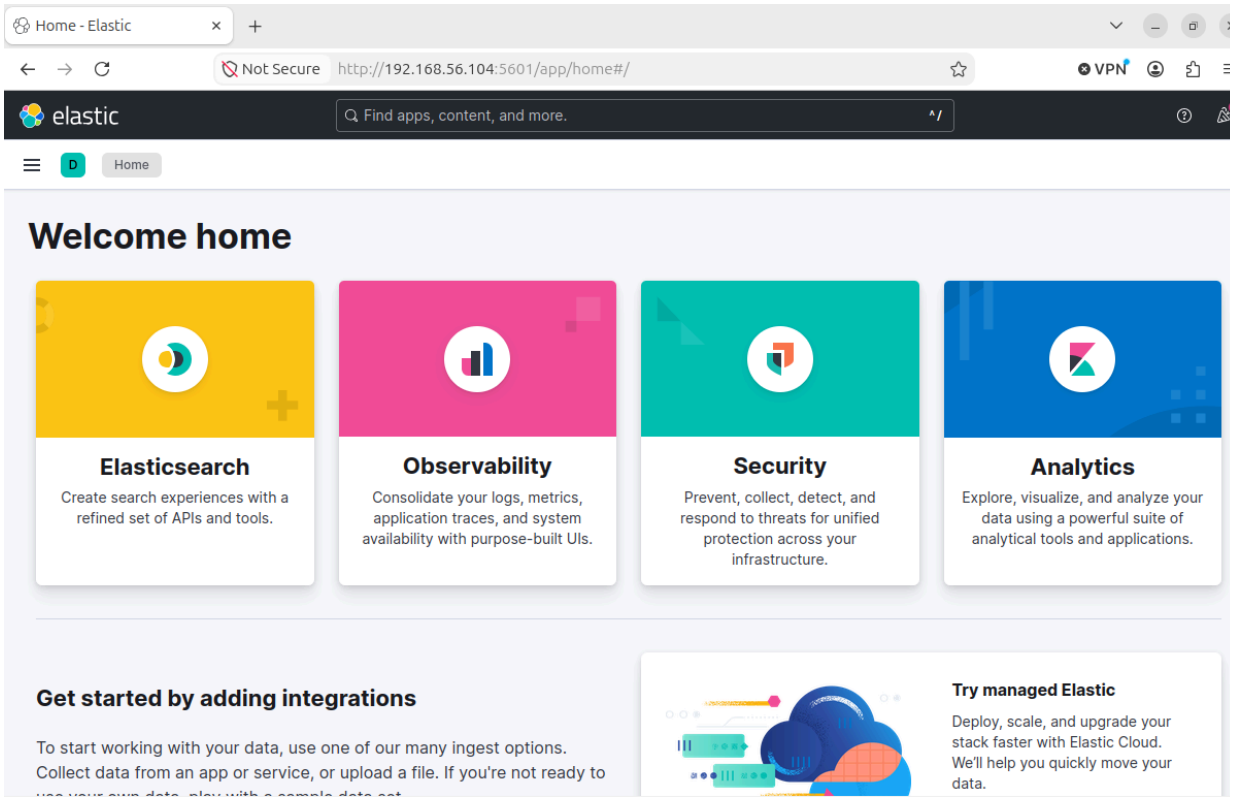
```
[charles-vence-espinas@centOS ~]$ curl -I http://192.168.56.108:9200
HTTP/1.1 200 OK
X-elastic-product: Elasticsearch
content-type: application/json
content-length: 541
```

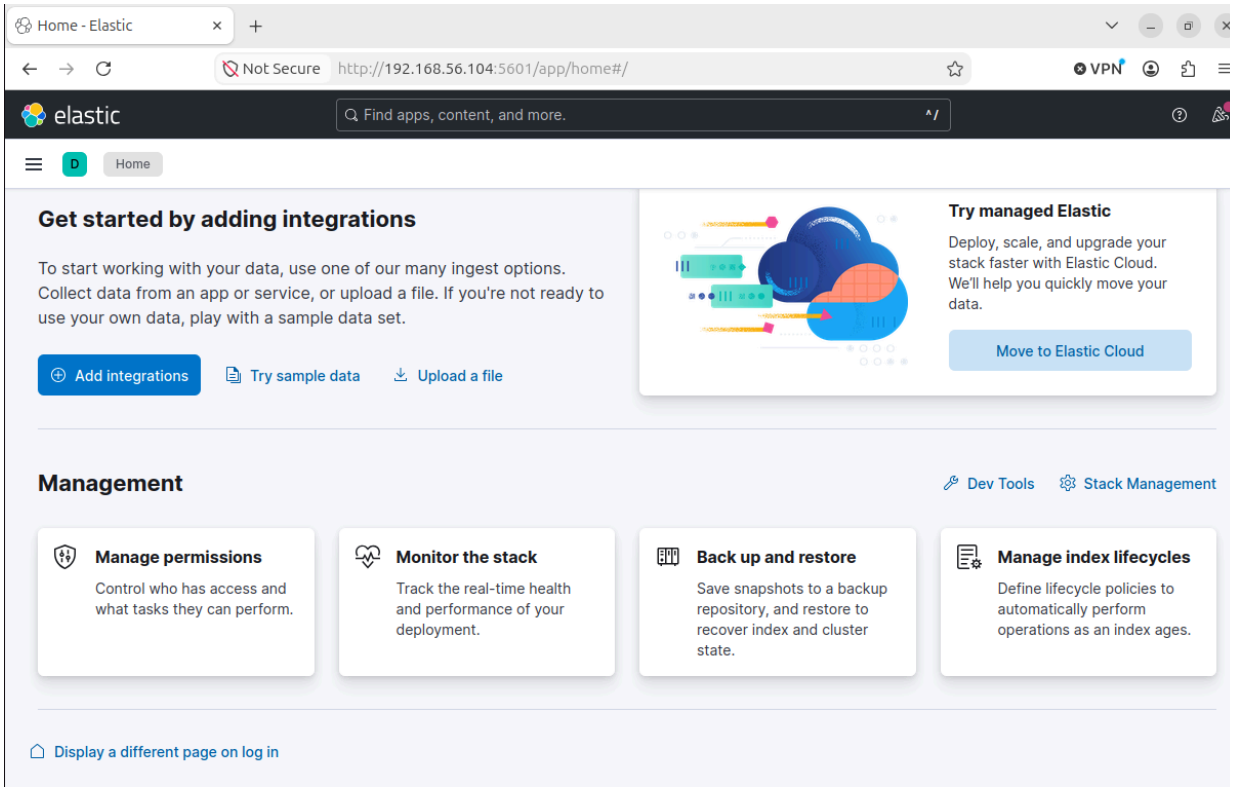
```
[charles-vence-espinas@centOS ~]$ curl -I http://192.168.56.104:5601
HTTP/1.1 302 Found
location: /spaces/enter
x-content-type-options: nosniff
referrer-policy: strict-origin-when-cross-origin
permissions-policy: camera=(), display-capture=(), fullscreen=(self), geolocation=(), microphone=(), web-share=()
cross-origin-opener-policy: same-origin
content-security-policy: script-src 'report-sample' 'self'; worker-src 'report-sample' 'self' blob:; style-src 'report-sample' 'self' 'unsafe-inline'
content-security-policy-report-only: form-action 'report-sample' 'self'; default-src 'report-sample' 'none'; font-src 'report-sample' 'self'; img-src 'report
sample' 'self' data: tiles.maps.elastic.co; connect-src 'report-sample' 'self' telemetry.elastic.co telemetry-staging.elastic.co feeds.elastic.co tiles.maps.
lastic.co vector.maps.elastic.co
kbn-name: centOS
kbn-license-sig: 1ff716af861b0aa12418628bcff7d7109437a05da69dcbf6f5485dbf743957d2
cache-control: private, no-cache, no-store, must-revalidate
content-length: 0
Date: Fri, 02 Oct 2026 12:45:31 GMT
Connection: keep-alive
Keep-Alive: timeout=120
```

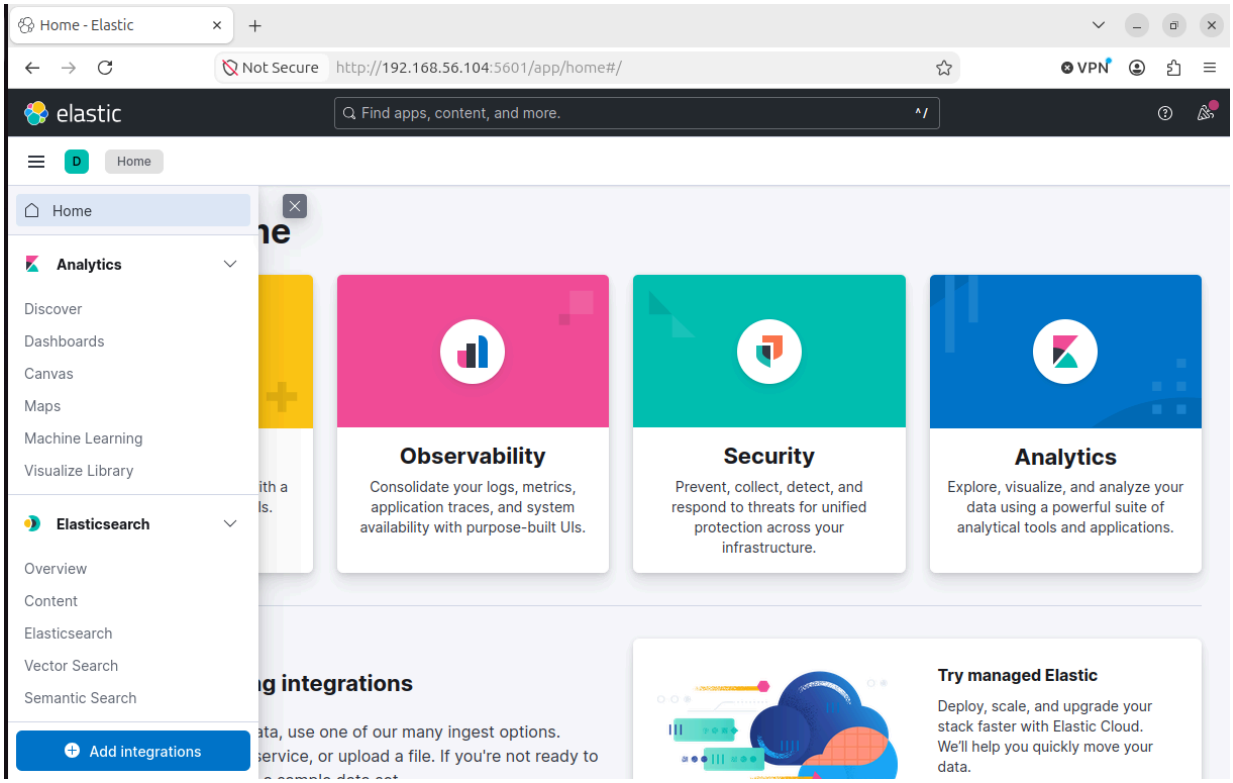
- Verification on centOS checks the web interface and network connection between different operating systems. The systemctl status command confirms that the kibana service is active and running. Querying http://192.168.56.108:9200 using curl verifies that CentOS firewalld allows connections to the Elasticsearch node on server1. Querying http://192.168.56.104:5601 also confirms that Kibana is listening on port 5601 and returning an HTTP/1.1 302 Found redirect response which shows that the web service is working properly.

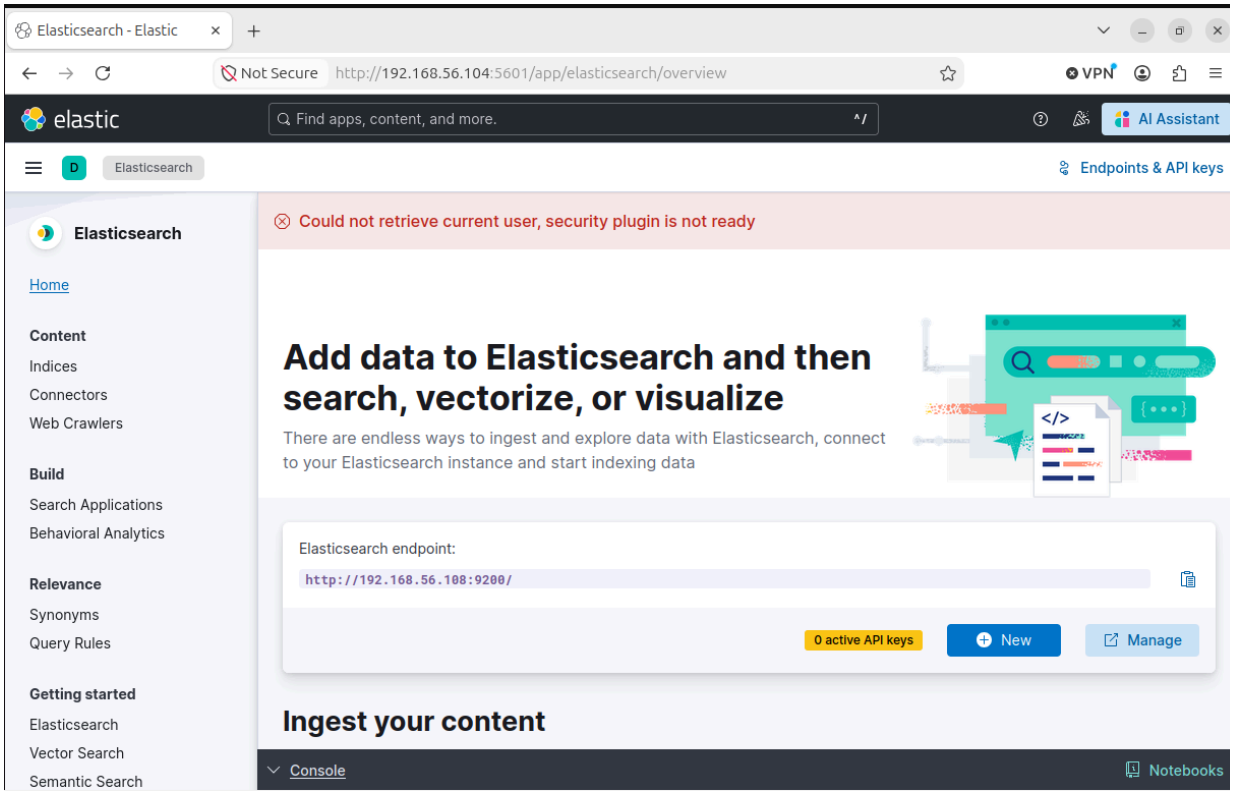
workstation

```
charles-vence-espinas@workstation:~/CPE212_CharlesVenceEspinas/ansible-lab/HOA10$ curl -I http://192.168.56.104:5601
HTTP/1.1 302 Found
location: /spaces/enter
x-content-type-options: nosniff
referrer-policy: strict-origin-when-cross-origin
permissions-policy: camera=(), display-capture=(), fullscreen=(self), geolocation=(), microphone=(), web-share=()
cross-origin-opener-policy: same-origin
content-security-policy: script-src 'report-sample' 'self'; worker-src 'report-sample' 'self' blob:; style-src 'report-s
ample' 'self' 'unsafe-inline'
content-security-policy-report-only: form-action 'report-sample' 'self'; default-src 'report-sample' 'none'; font-src '
eport-sample' 'self'; img-src 'report-sample' 'self' data: tiles.maps.elastic.co; connect-src 'report-sample' 'self' te
emetry.elastic.co telemetry-staging.elastic.co feeds.elastic.co tiles.maps.elastic.co vector.maps.elastic.co
kbn-name: centOS
kbn-license-sig: 1ff716af861b0aa12418628bcff7d7109437a05da69dcbf6f5485dbf743957d2
cache-control: private, no-cache, no-store, must-revalidate
content-length: 0
Date: Fri, 02 Oct 2026 12:41:50 GMT
Connection: keep-alive
Keep-Alive: timeout=120
```




Home - Elastic
Not Secure
http://192.168.56.104:5601/app/home#/
VPN
elastic
Find apps, content, and more.
Home
Get started by adding integrations
To start working with your data, use one of our many ingest options. Collect data from an app or service, or upload a file. If you're not ready to use your own data, play with a sample data set.
Add integrations
Try sample data
Upload a file
Try managed Elastic
Deploy, scale, and upgrade your stack faster with Elastic Cloud. We'll help you quickly move your data.
Move to Elastic Cloud
Management
Dev Tools
Stack Management
Manage permissions
Control who has access and what tasks they can perform.
Monitor the stack
Track the real-time health and performance of your deployment.
Back up and restore
Save snapshots to a backup repository, and restore to recover index and cluster state.
Manage index lifecycles
Define lifecycle policies to automatically perform operations as an index ages.
Display a different page on log in


Home - Elastic
Not Secure
http://192.168.56.104:5601/app/home#/
VPN
elastic
Find apps, content, and more.
Home
Home
Analytics
Discover
Dashboards
Canvas
Maps
Machine Learning
Visualize Library
Elasticsearch
Overview
Content
Elasticsearch
Vector Search
Semantic Search
Add integrations
Observability
Consolidate your logs, metrics, application traces, and system availability with purpose-built UIs.
Security
Prevent, collect, detect, and respond to threats for unified protection across your infrastructure.
Analytics
Explore, visualize, and analyze your data using a powerful suite of analytical tools and applications.
Try managed Elastic
Deploy, scale, and upgrade your stack faster with Elastic Cloud. We'll help you quickly move your data.



- Verification from the workstation confirms that Kibana can be accessed through both command-line testing and a web browser. Running curl -I http://192.168.56.104:5601 in the workstation terminal returns an HTTP/1.1 302 Found response which confirms that the CentOS firewalld rules allow incoming TCP traffic through the host-only network interface (enp0s8). Opening http://192.168.56.104:5601 in the workstation web browser successfully displays the Kibana graphical user interface ("Welcome home" dashboard). This confirms that Kibana is listening on all network interfaces (0.0.0.0:5601) and communicating with the backend Elasticsearch cluster on server1 (192.168.56.108:9200). Accessing Stack Management and Elasticsearch Overview in the browser also displays healthy cluster metrics which confirms the successful integration of all three nodes.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Reflections

Answer the following:

1. What are the benefits of having log monitoring tool?

- Having a log monitoring tool provides several benefits which include centralized log management, real time monitoring, faster troubleshooting and improved security. It helps detect system errors, performance issues and suspicious activities which allows administrators to respond to problems before they affect users. It also provides automated alerts, supports compliance and auditing and helps track system performance and resource usage which makes IT infrastructure easier to manage and maintain.

Conclusions

Through this hands on activity, I learned how to automate the deployment and integration of a multi node Elastic Stack across different Linux environments using Ansible roles and Jinja2 dynamic templating. I learned how to set up dedicated Virtual Machines for Elasticsearch, Logstash, and Kibana which work together to provide centralized log collection and visualization. I also learned how to configure memory heap limits, swap files, and firewall rules

which help maintain system stability and allow secure communication between nodes. By performing systemd service checks, command line HTTP queries, and browser UI testing, I was able to verify that all components were working properly. Overall, I learned how Infrastructure as Code can simplify complex deployments, reduce manual configuration errors and make system management more efficient.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Code Assistance**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I use this tool to help me in a situation where I am writing a code and having difficulty**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.